

Interfaçage API REST, JSON et Webhooks

Ce document détaille comment PowerShell interagit avec les services web modernes et les plateformes Cloud. Dans un écosystème DevOps (CI/CD, Cloud public, supervision), l'administration ne se fait plus seulement via des modules fermés, mais en appelant directement des API REST et en échangeant des données au format JSON.

1. Le Format JSON : Le Langage Universel du Cloud

Historiquement, l'administration système utilisait des fichiers texte, des fichiers INI ou du XML (très verbeux). Aujourd'hui, l'écrasante majorité des API (GitHub, GitLab, AWS, Azure, Slack) communiquent en **JSON (JavaScript Object Notation)**.

C'est un format textuel léger, structuré en paires de clé/valeur, et facilement lisible par un humain.

1.1 Convertir vers JSON (Sérialisation)

Pour envoyer des données PowerShell vers une API, il faut d'abord transformer notre objet [PSCustomObject] ou notre Table de Hachage @{ } en chaîne de caractères JSON à l'aide de **ConvertTo-Json**.

```
$ServeurInfo = [PSCustomObject]@{  
    Nom      = "SRV-WEB-01"  
    Role     = "IIS"  
    Actif    = $true  
    AdressesIP = @("10.0.0.10", "192.168.1.10")  
}
```

```
# Sérialisation en JSON  
$PayloadJson = $ServeurInfo | ConvertTo-Json  
Write-Host $PayloadJson
```

Sortie JSON :

```
{  
  "Nom": "SRV-WEB-01",  
  "Role": "IIS",  
  "Actif": true,  
  "AdressesIP": [  
    "10.0.0.10",
```

```
"192.168.1.10"  
]  
}
```

Le piège classique : le paramètre -Depth

Par défaut, ConvertTo-Json ne descend qu'à 2 niveaux de profondeur. Si vous sérialisez un objet très complexe (avec des objets imbriqués dans d'autres objets), PowerShell tronquera les données et affichera le type au lieu de la valeur. Il faut toujours forcer la profondeur sur les objets complexes :

```
ConvertTo-Json -Depth 10
```

1.2 Convertir depuis JSON (Désérialisation)

À l'inverse, lorsqu'une API vous renvoie du JSON, vous devez le retransformer en objet PowerShell pour pouvoir le manipuler. C'est le rôle de **ConvertFrom-Json**.

```
$JsonBrut = '{"Serveur": "SRV-BDD", "DisqueLibreGo": 45.5}'  
$ObjetPowerShell = $JsonBrut | ConvertFrom-Json
```

```
# On peut désormais interagir avec lui comme un objet normal :  
Write-Host "Le serveur $($ObjetPowerShell.Serveur) a $($ObjetPowerShell.DisqueLibreGo) Go  
libres."
```

2. Consommer une API REST (Lecture - Méthode GET)

La cmdlet historique Invoke-WebRequest télécharge du contenu web brut (comme un navigateur). En automatisation, on privilégie **Invoke-RestMethod**.

La force de Invoke-RestMethod, c'est qu'elle appelle l'API et exécute automatiquement un ConvertFrom-Json sur la réponse. Le résultat est directement un objet PowerShell exploitable.

2.1 Exemple : Interrogation d'une API publique (sans authentification)

```
# Interroger une API fictive renvoyant des utilisateurs  
$Uri = "https://jsonplaceholder.typicode.com/users"  
$Utilisateurs = Invoke-RestMethod -Uri $Uri -Method Get
```

```
# Affichage natif PowerShell  
$Utilisateurs | Select-Object name, email, website | Format-Table -AutoSize
```

3. L'Authentification auprès des API

La plupart des API d'entreprise exigent une preuve d'identité. Cette preuve est généralement transmise dans les **En-têtes (Headers)** de la requête HTTP.

(Note : On utilise ici le module SecretStore vu au J4S3 pour ne pas écrire le token en clair).

3.1 Authentification par Jeton (Bearer Token)

C'est le standard actuel (OAuth2, GitLab, Microsoft Graph).

1. Récupération sécurisée du token (sans l'afficher)

```
$MonJeton = Get-Secret -Name "Token_GitLab_API" -AsPlainText
```

2. Construction de l'en-tête HTTP

```
$Headers = @{  
    "Authorization" = "Bearer $MonJeton"  
    "Accept"       = "application/json"  
}
```

3. Exécution de la requête avec les en-têtes

```
$Projets = Invoke-RestMethod -Uri "https://gitlab.entreprise.local/api/v4/projects" -Headers  
$Headers -Method Get
```

4. Purge immédiate de la variable sensible

```
$MonJeton = $null
```

3.2 Authentification Basique (Basic Auth)

Souvent utilisée par les API plus anciennes, elle requiert l'encodage de l'identifiant et du mot de passe en Base64, ou simplement le passage d'un objet [PSCredential].

```
$Creds = Get-Secret -Name "Cred_API_Legacy"
```

```
$Resultat = Invoke-RestMethod -Uri "https://api.vieux-systeme.local/data" -Credential $Creds  
-Method Get
```

4. Modifier des Données (Création - Méthode POST / PUT)

Pour créer une ressource sur un système distant (créer un ticket Jira, déclencher un Job CI/CD, créer un utilisateur Microsoft 365), on utilise généralement la méthode **POST**.

Il faut fournir à l'API un "corps" de requête (le Payload) formaté en JSON, et lui préciser que l'on

parle ce langage via le paramètre -ContentType.

1. Définition des données sous forme d'objet PowerShell

```
$NouveauTicket = @{  
    titre      = "Alerte Espace Disque"  
    description = "Le serveur SRV-FILE01 a moins de 10% d'espace libre."  
    urgence    = "Haute"  
}
```

2. Sérialisation en JSON

```
$Payload =$NouveauTicket | ConvertTo-Json
```

3. Envoi via la méthode POST

```
$Reponse = Invoke-RestMethod -Uri "https://api.ticketing.local/v1/incidents" `
    -Method Post `
    -Body $Payload `
    -ContentType "application/json" `
    -Headers $Headers
```

```
Write-Host "Ticket créé avec l'ID : $($Reponse.ticket_id)"
```

5. Les Webhooks : La Communication Événementielle

Un Webhook est une URL unique fournie par une application (comme Teams, Slack, Discord, GitLab) qui écoute passivement. Lorsqu'elle reçoit une requête POST HTTP avec des données formatées spécifiquement, elle déclenche une action (comme poster un message dans un canal).

C'est l'outil privilégié pour notifier les équipes de la réussite ou de l'échec d'un déploiement PowerShell automatisé.

5.1 Exemple : Envoyer une notification Microsoft Teams ou Slack

```
$WebhookURL = "https://hooks.slack.com/services/T000/B000/XXXX"
```

```
$Message = @{  
    text = "🚀 Déploiement du serveur SRV-WEB terminé avec succès par l'automate  
PowerShell."  
}
```

```
Invoke-RestMethod -Uri $WebhookURL -Method Post -Body ($Message | ConvertTo-Json)  
-ContentType "application/json"
```

6. Gérer les Erreurs HTTP et Résilience

Par défaut, si une API renvoie un code d'erreur HTTP (ex: 401 Unauthorized, 404 Not Found, 500 Internal Server Error), Invoke-RestMethod lève une **erreur terminante** (rouge bloquante).

Il est indispensable de les capturer avec un bloc try/catch pour rendre votre script de déploiement robuste.

```
try {
    $Resultat = Invoke-RestMethod -Uri "https://api.entreprise.local/status" -Method Get
-ErrorAction Stop
}
catch {
    Write-Warning "L'API a refusé la connexion ou est injoignable."

    # Extraire le message technique de l'exception
    $MessageErreur = $_.Exception.Message
    Write-Host "Détail : $MessageErreur" -ForegroundColor Red

    # En contexte CI/CD, on force le script à échouer avec exit 1
    exit 1
}
```

7. Le Cas des Certificats Auto-Signés en Développement

Dans des environnements de test, il arrive que les API exposent du HTTPS avec un certificat non reconnu (auto-signé). PowerShell refusera la connexion par sécurité.

Depuis PowerShell 6+, le commutateur -SkipCertificateCheck a été ajouté pour ignorer volontairement cette alerte :

```
# Utile uniquement en LAB. À proscrire en production absolue.
Invoke-RestMethod -Uri "https://10.0.0.50/api" -Method Get -SkipCertificateCheck
```